Lecture 5: t-SNE, deep neural network-based representation learning

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 6, 2026

Announcements

- HW1 due 4/9
- Tutorial 3 (neural networks) 4/8
- Quiz 1 4/13

Plan for today

More nonlinear dimensionality reduction

- t-SNE
- Neural network-based methods

t-Distributed Stochastic Neighborhood Embedding (t-SNE)

If our goal is **visualization**...

- We really only care about reduction to $k = 2$ or 3
- We don't need an out-of-sample extension

t-Distributed Stochastic Neighborhood Embedding (t-SNE)

If our goal is **visualization**...

- We really only care about reduction to $k = 2$ or 3
- We don't need an out-of-sample extension

t-SNE: a nonlinear dimensionality reduction method that is particularly good in this setting [van der Maaten & Hinton 2008]

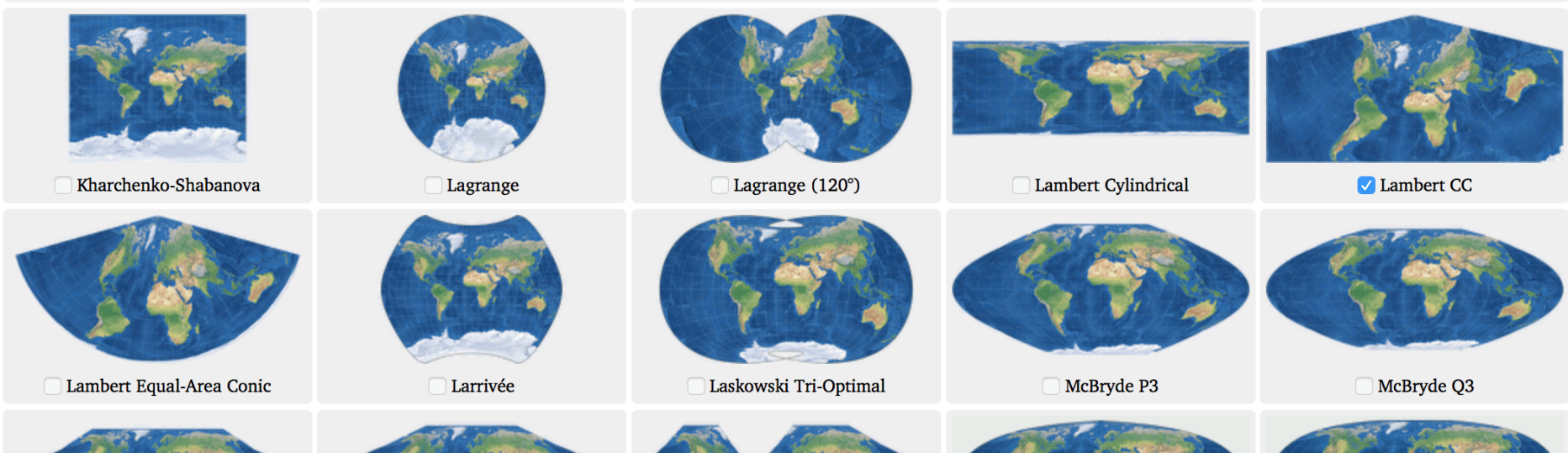
- Extremely widely used
- Gives us opportunity to discuss some useful information-theoretic concepts

Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- Most nonlinear methods we have covered try to maintain similar distances/inner products between points in original space $\mathbf{x}$ and in low-dimensional projections $\mathbf{y}$
- E.g. MDS: $\min \sum_{ij} (\mathbf{x}_i \cdot \mathbf{x}_j - \mathbf{y}_i \cdot \mathbf{y}_j)^2$
- (Note: change of notation from $\mathbf{z}$ in previous lectures to $\mathbf{y}$, to match the literature related to today's topic.)
- But it's impossible to actually maintain distances/similarities when we squash to only 2-3 dimensions!

Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- Most nonlinear methods we have covered try to maintain similar distances/inner products between points in original space $\mathbf{x}$ and in low-dimensional projections $\mathbf{y}$
- E.g. MDS: $\min \sum_{ij} (\mathbf{x}_i \cdot \mathbf{x}_j - \mathbf{y}_i \cdot \mathbf{y}_j)^2$
- (Note: change of notation from $\mathbf{z}$ in previous lectures to $\mathbf{y}$, to match the literature related to today's topic.)
- But it's impossible to actually maintain distances/similarities when we squash to only 2-3 dimensions!



Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- Most nonlinear methods we have covered try to maintain similar distances/inner products between points in original space $\mathbf{x}$ and in low-dimensional projections $\mathbf{y}$
- E.g. MDS: $\min \sum_{ij} (\mathbf{x}_i \cdot \mathbf{x}_j - \mathbf{y}_i \cdot \mathbf{y}_j)^2$
- (Note: change of notation from $\mathbf{z}$ in previous lectures to $\mathbf{y}$, to match the literature related to today's topic.)
- But it's impossible to actually maintain distances/similarities when we squash to only 2-3 dimensions!
- So let's not even try
- Instead try to maintain *neighborhood properties*: If one point is likely to be a near neighbor of another in the original space, it should also be a near neighbor in the low-dim space

Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- If $\mathbf{x}_i, \mathbf{x}_j$ are likely to be neighbors in the original space, then their mappings in the low-dim space $\mathbf{y}_i, \mathbf{y}_j$ should also be neighbors
- SNE formalizes this: Define the probability that point $\mathbf{x}_i$ would pick $\mathbf{x}_j$ as its neighbor as

$$p_{j|i} = \begin{cases} \frac{e^{(-||\mathbf{x}_i - \mathbf{x}_j||^2)/2\sigma_i^2}}{\sum_{k \neq i} e^{(-||\mathbf{x}_i - \mathbf{x}_k||^2)/2\sigma_i^2}} & \text{if } i \neq j \\ 0 & \text{if } i = j \end{cases}$$

where σ_i is a data point-specific variance of Gaussian centered at $\mathbf{x}_i$.

Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- If $\mathbf{x}_i, \mathbf{x}_j$ are likely to be neighbors in the original space, then their mappings in the low-dim space $\mathbf{y}_i, \mathbf{y}_j$ should also be neighbors
- SNE formalizes this: Define the probability that point $\mathbf{x}_i$ would pick $\mathbf{x}_j$ as its neighbor as

$$p_{j|i} = \begin{cases} \frac{e^{(-||\mathbf{x}_i - \mathbf{x}_j||^2)/2\sigma_i^2}}{\sum_{k \neq i} e^{(-||\mathbf{x}_i - \mathbf{x}_k||^2)/2\sigma_i^2}} & \text{if } i \neq j \\ 0 & \text{if } i = j \end{cases}$$

  where σ_i is a data point-specific variance of Gaussian centered at $\mathbf{x}_i$.
- And in the low-dimensional space:

$$q_{j|i} = \begin{cases} \frac{e^{(-||\mathbf{y}_i - \mathbf{y}_j||^2)}}{\sum_{k \neq i} e^{(-||\mathbf{y}_i - \mathbf{y}_k||^2)}} & \text{if } i \neq j \\ 0 & \text{if } i = j \end{cases}$$

  (variance fixed in low-dim space—changing it just rescales mapping)

Stochastic Neighborhood Embedding (SNE) [Hinton & Roweis 2002]

- Let P_i (Q_i) represent conditional distribution over all other data points given $\mathbf{x}_i$ ($\mathbf{y}_i$)
- Objective to minimize is a sum over KL divergences:

$$\sum_i \mathrm{KL}(P_i||Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

- Large cost for using widely separated $\mathbf{y}_i, \mathbf{y}_j$ (small $q_{j|i}$) to represent nearby $\mathbf{x}_i, \mathbf{x}_j$ (large $p_{j|i}$)
- Objective is nonconvex, optimized with gradient descent (somewhat involved)

Sidebar: Kullback-Leibler (KL) divergence

But first, variable-length coding:

Char	Freq	Code
space	7	111
a	4	010
e	4	000
f	3	1101
h	2	1010
i	2	1000
m	2	0111
n	2	0010
s	2	1011
t	2	0110
l	1	11001
o	1	00110
p	1	10011
r	1	11000
u	1	00111
x	1	10010

[https://en.wikipedia.org/wiki/Huffman_coding]

Sidebar: Kullback-Leibler (KL) divergence

$$\begin{aligned} \mathrm{KL}(P||Q) &= \sum_j p_j \log \frac{p_j}{q_j} \\ &= H(P,Q) - H(P) = -\mathrm{E}_P[\log Q] + \mathrm{E}_P[\log P] \end{aligned}$$

where $H(P) =$ entropy of P in bits, $H(P,Q) =$ cross entropy between P and Q in bits. Interpretations:

Sidebar: Kullback-Leibler (KL) divergence

$$\begin{aligned} \mathrm{KL}(P||Q) &= \sum_j p_j \log \frac{p_j}{q_j} \\ &= H(P,Q) - H(P) = -\mathrm{E}_P[\log Q] + \mathrm{E}_P[\log P] \end{aligned}$$

where $H(P) =$ entropy of P in bits, $H(P,Q) =$ cross entropy between P and Q in bits. Interpretations:

- Entropy = number of bits, on average, needed to optimally encode samples of the random variable

Sidebar: Kullback-Leibler (KL) divergence

$$\begin{aligned} \mathrm{KL}(P||Q) &= \sum_j p_j \log \frac{p_j}{q_j} \\ &= H(P,Q) - H(P) = -\mathrm{E}_P[\log Q] + \mathrm{E}_P[\log P] \end{aligned}$$

where $H(P) =$ entropy of P in bits, $H(P,Q) =$ cross entropy between P and Q in bits. Interpretations:

- Entropy = number of bits, on average, needed to optimally encode samples of the random variable
- Cross entropy = number of bits, on average, needed to encode samples from P using a code optimized for Q (where "optimized" is in the sense of minimum expected length)

Sidebar: Kullback-Leibler (KL) divergence

$$\begin{aligned} \mathrm{KL}(P||Q) &= \sum_j p_j \log \frac{p_j}{q_j} \\ &= H(P,Q) - H(P) = -\mathrm{E}_P[\log Q] + \mathrm{E}_P[\log P] \end{aligned}$$

where $H(P) =$ entropy of P in bits, $H(P,Q) =$ cross entropy between P and Q in bits. Interpretations:

- Entropy = number of bits, on average, needed to optimally encode samples of the random variable
- Cross entropy = number of bits, on average, needed to encode samples from P using a code optimized for Q (where "optimized" is in the sense of minimum expected length)
- KL divergence = number of extra bits, on average, needed to encode samples from P using a code optimized for Q

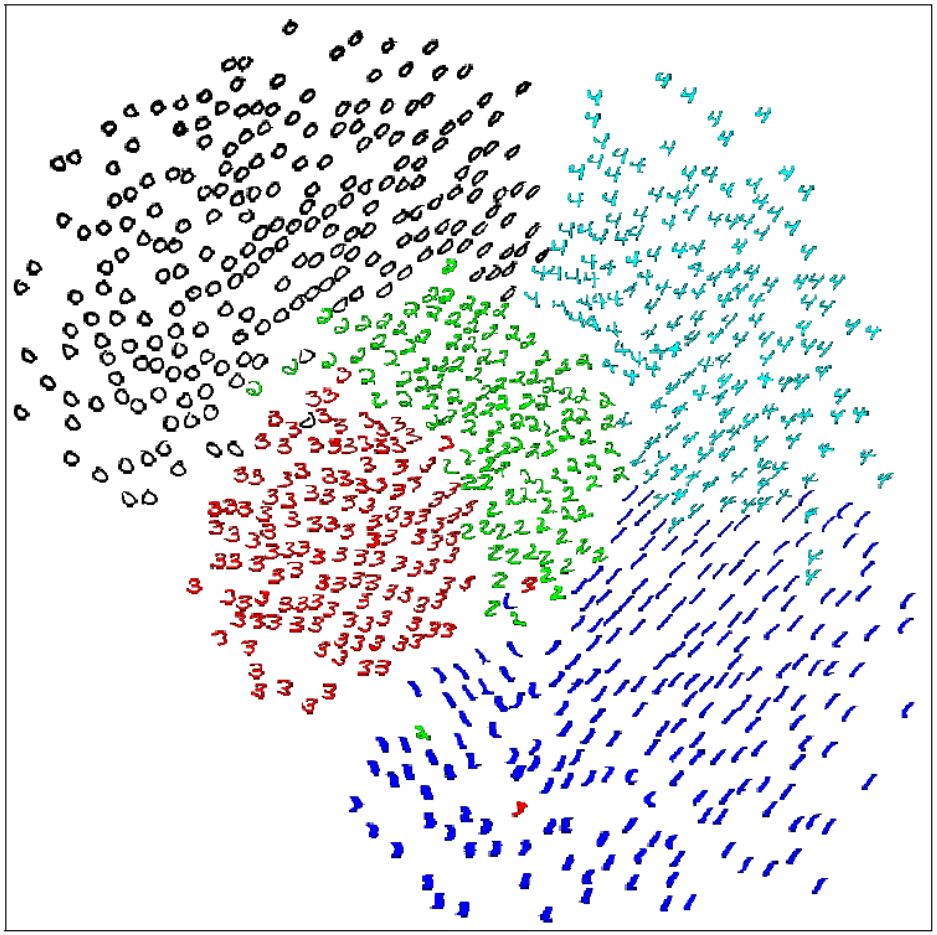
Sidebar: Kullback-Leibler (KL) divergence

$$\begin{aligned} \mathrm{KL}(P||Q) &= \sum_j p_j \log \frac{p_j}{q_j} \\ &= H(P,Q) - H(P) = -\mathrm{E}_P[\log Q] + \mathrm{E}_P[\log P] \end{aligned}$$

where $H(P) =$ entropy of P in bits, $H(P,Q) =$ cross entropy between P and Q in bits. Interpretations:

- Entropy = number of bits, on average, needed to optimally encode samples of the random variable
- Cross entropy = number of bits, on average, needed to encode samples from P using a code optimized for Q (where "optimized" is in the sense of minimum expected length)
- KL divergence = number of extra bits, on average, needed to encode samples from P using a code optimized for Q
- = Amount of information loss when approximating P with Q

SNE: MNIST example



[Hinton & Roweis 2002]

t-SNE [van der Maaten & Hinton 2008]

- SNE is difficult to optimize, and suffers from a "crowding problem"
- t-SNE makes two clever engineering improvements:
  - Uses a symmetrized version of the SNE cost with simpler gradients that are faster to compute
  - Uses a heavier-tailed Student-t distribution instead of Gaussian in the low-dimensional space

From SNE to t-SNE

Symmetrized SNE

- Uses **joint** instead of conditional distributions of data pairs: minimizes

$$\mathrm{KL}(P||Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

(where $p_{ii} = q_{ii} = 0$)

- In low-dim space:

$$q_{ij} = \frac{e^{(-||\mathbf{y}_i - \mathbf{y}_j||^2)}}{\sum_{k \neq l} e^{(-||\mathbf{y}_k - \mathbf{y}_l||^2)}}$$

From SNE to t-SNE

Symmetrized SNE

- In high-dim space, it would seem natural to define

$$p_{ij} = \frac{e^{(-||\mathbf{x}_i - \mathbf{x}_j||^2)/2\sigma^2}}{\sum_{k \neq l} e^{(-||\mathbf{x}_k - \mathbf{x}_l||^2)/2\sigma^2}}$$

- But this doesn't allow data point-specific σ_i
- Instead, symmetrized SNE uses

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2}$$

From SNE to t-SNE

- Because the Gaussian distribution dies off very fast with distance from the mean, SNE only accurately represents very nearby points
- Moderately distant points can't be differentiated from each other
- Many points end up "crushed" into the center of the map
- t-SNE solution: use a heavy-tailed distribution in the low-dim space

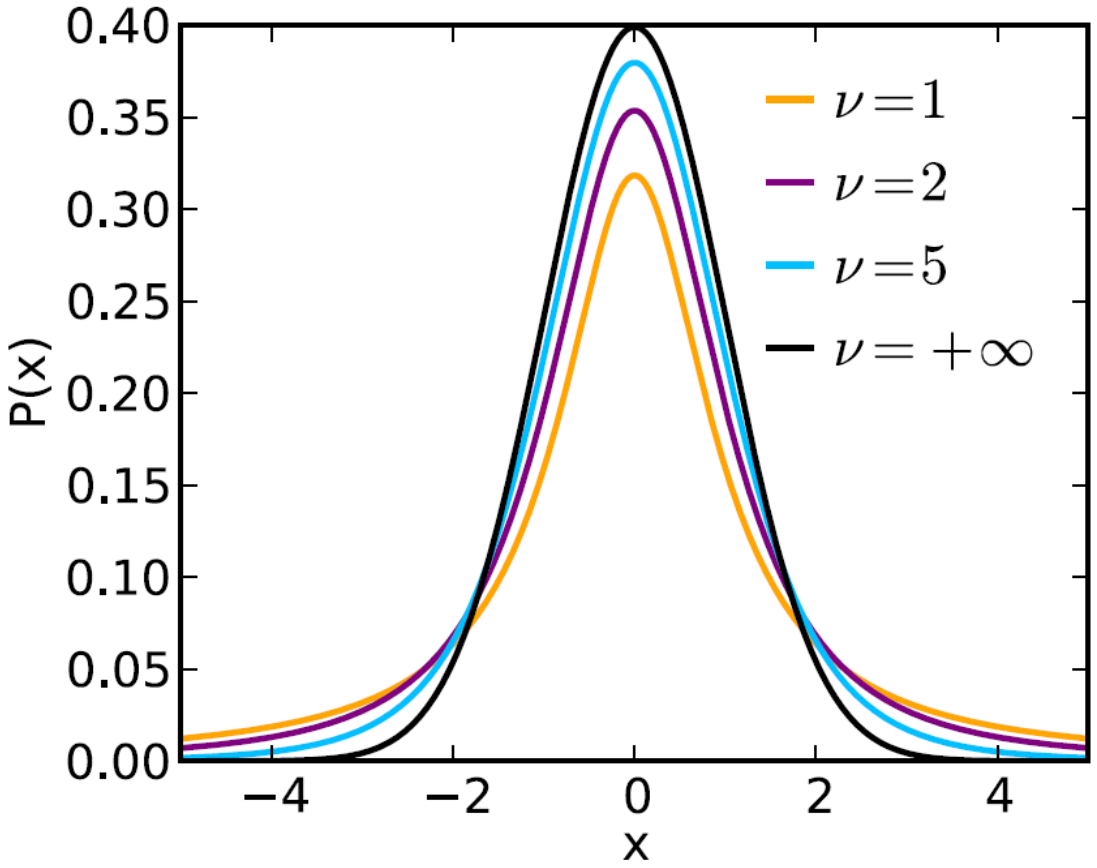
t-SNE

- p_{ij} is as in symmetrized SNE
- In low-dim space, use a Student t-distribution with one degree of freedom:

$$q_{ij} = \frac{(1 + ||\mathbf{y}_i - \mathbf{y}_j||^2)^{-1}}{\sum_{k \neq l}(1 + ||\mathbf{y}_k - \mathbf{y}_l||^2)^{-1}}$$

Student's t-distribution

Densities for varying number of degrees of freedom ν



[Wikipedia]

Examples: MNIST visualized with Isomap



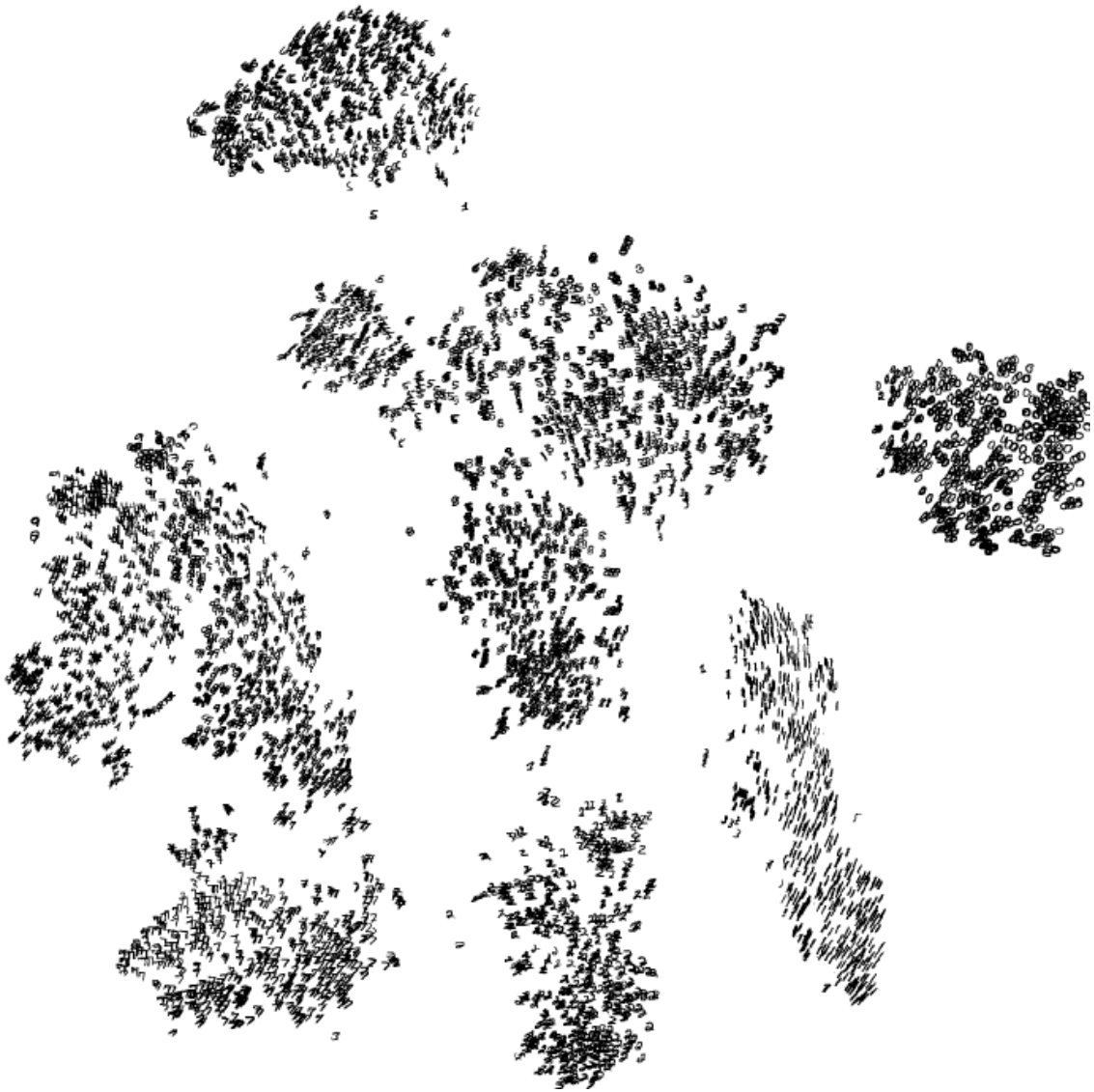
[van der Maaten & Hinton 2008 supp.]

Examples: MNIST visualized with SNE



[van der Maaten & Hinton 2008 supp.]

Examples: MNIST visualized with t-SNE



[van der Maaten & Hinton 2008 supp.]

Examples: COIL-20 rotated object data set visualized with SNE



[van der Maaten & Hinton 2008 supp.]

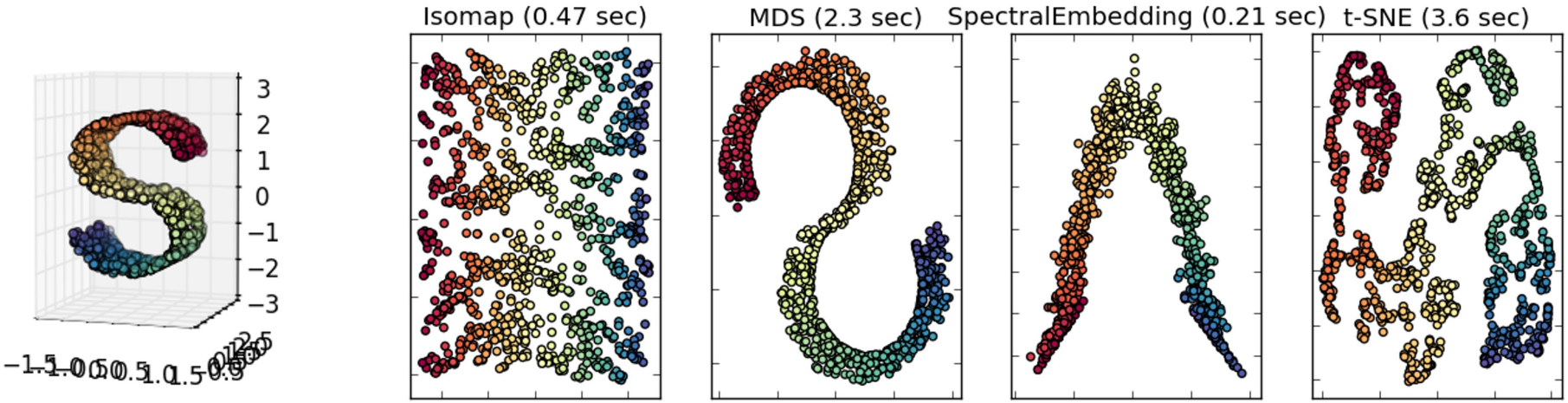
Examples: COIL-20 rotated object data set visualized with t-SNE



[van der Maaten & Hinton 2008 supp.]

t-SNE examples ☹

Tends to fail for Swiss roll-type toy data:



[http://scikit-learn.org/stable/modules/manifold.html]

Setting the hyperparameters σ_i

- Data point-specific variances allow SNE/t-SNE to model varying densities in different parts of the high-dim space
- But wait, that's as many hyperparameters as data points!
- SNE/t-SNE sets σ_i based on a single user-provided hyperparameter, **perplexity**
- The perplexity of the distribution P_i is

$$\begin{aligned} \mathrm{Perp}(P_i) &= 2^{H(P_i)} \\ H(P_i) &= -\sum_j p_{j|i} \log_2 p_{j|i} \end{aligned}$$

- $H(P_i)$ is the entropy of P_i (measured in bits)

Setting the hyperparameters σ_i

- Data point-specific variances allow SNE/t-SNE to model varying densities in different parts of the high-dim space
- But wait, that's as many hyperparameters as data points!
- SNE/t-SNE sets σ_i based on a single user-provided hyperparameter, **perplexity**
- The perplexity of the distribution P_i is

$$\begin{aligned} \mathrm{Perp}(P_i) &= 2^{H(P_i)} \\ H(P_i) &= -\sum_j p_{j|i} \log_2 p_{j|i} \end{aligned}$$

- $H(P_i)$ is the entropy of P_i (measured in bits)
- Note: There are multiple related meanings of "perplexity" (we'll see a different one for language models later)

Sidebar: Perplexity

- Entropy = average number of bits needed to optimally encode samples of the random variable
- Perplexity = "effective number of values" of the random variable
- Why?

Sidebar: Perplexity

- Entropy = average number of bits needed to optimally encode samples of the random variable
- Perplexity = "effective number of values" of the random variable
- Why?
- Perplexity of a uniform distribution P over N values:

$$\begin{aligned} \mathrm{Perp}(P) &= 2^{-\sum_i p_i \log_2 p_i} \\ &= 2^{-\sum_i \frac{1}{N} \log_2 \left(\frac{1}{N}\right)} \end{aligned}$$

Sidebar: Perplexity

- Entropy = average number of bits needed to optimally encode samples of the random variable
- Perplexity = "effective number of values" of the random variable
- Why?
- Perplexity of a uniform distribution P over N values:

$$\begin{aligned} \mathrm{Perp}(P) &= 2^{-\sum_i p_i \log_2 p_i} \\ &= 2^{-\sum_i \frac{1}{N} \log_2 \left(\frac{1}{N}\right)} \\ &= 2^{\sum_i \frac{1}{N} \log_2 N} \end{aligned}$$

Sidebar: Perplexity

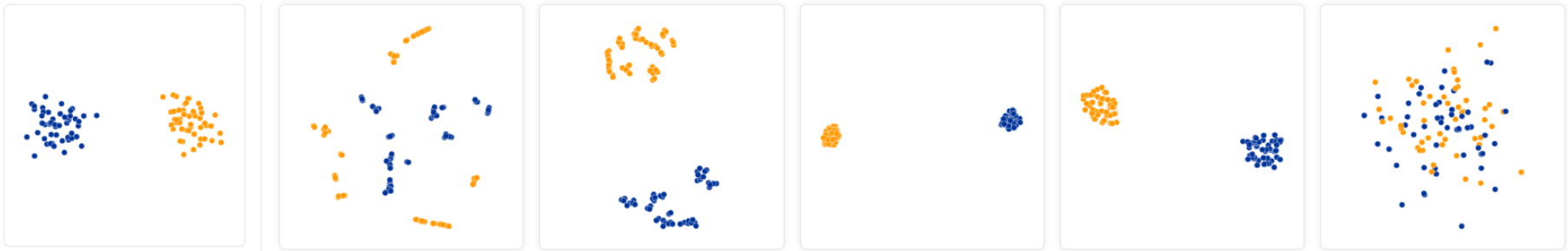
- Entropy = average number of bits needed to optimally encode samples of the random variable
- Perplexity = "effective number of values" of the random variable
- Why?
- Perplexity of a uniform distribution P over N values:

$$\begin{aligned}
\mathrm{Perp}(P) &= 2^{-\sum_i p_i \log_2 p_i} \\
&= 2^{-\sum_i \frac{1}{N} \log_2 \left(\frac{1}{N}\right)} \\
&= 2^{\sum_i \frac{1}{N} \log_2 N} \\
&= 2^{\log_2 N} = N
\end{aligned}$$

- Perplexity of a distribution P where one value has probability 1:

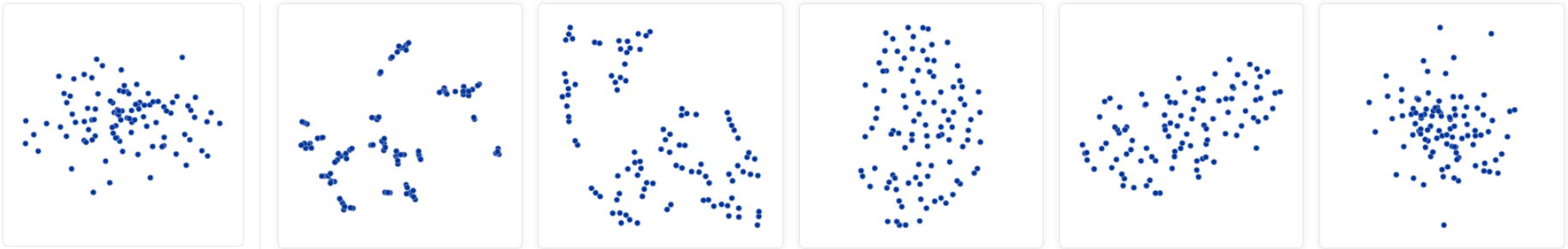
$$\begin{aligned}
\mathrm{Perp}(P) &= 2^{-\sum_i p_i \log_2 p_i} \\
&= 2^{-1 \log_2 1} = 2^0 = 1
\end{aligned}$$

t-SNE: varying perplexity



[http://distill.pub/2016/misread-tsne/]

t-SNE: varying perplexity



Original

Perplexity: 2
Step: 5,000

Perplexity: 5
Step: 5,000

Perplexity: 30
Step: 5,000

Perplexity: 50
Step: 5,000

Perplexity: 100
Step: 5,000

[http://distill.pub/2016/misread-tsne/]

t-SNE: summary

- Go-to technique for visualization
- Often used to visualize the results of other dimensionality reduction techniques (with higher dimensionality k)
- Works especially well for clustered data, or data with multiple "sub-manifolds"

t-SNE: summary

- Go-to technique for visualization
- Often used to visualize the results of other dimensionality reduction techniques (with higher dimensionality k)
- Works especially well for clustered data, or data with multiple "sub-manifolds"
- Caveats
  - For other types of data, may not work well (alternative technique: UMAP)
  - Perplexity hyperparameter is important
  - Careful how you interpret it: Don't trust visualizations with lots of tiny clusters, or just one big blob
  - Note: t-SNE is non-parametric and has no built-in out-of-sample extension; a parametric extension exists (parametric t-SNE – more on this later)

(Deep) neural network-based methods

Feature functions

- Machine learning relies almost entirely on linear predictors
- But often applied to non-linear *features* of the data
- Feature transform: $\boldsymbol{\phi} : \mathcal{X} \to \mathbb{R}^d$

$$f_y(\mathbf{x}; \mathbf{w}, \mathbf{b}) = \mathbf{w}_y \cdot \boldsymbol{\phi}(\mathbf{x}) + b_y$$

- Shallow learning: hand-crafted, non-hierarchical $\boldsymbol{\phi}$.
- Basic example: polynomial regression, $\phi_j(x) = x^j$, $j = 0, \ldots, d$, $\hat{y} = \mathbf{w} \cdot \boldsymbol{\phi}(x)$
- Kernel-based learning: employing kernel K corresponds to (some) feature space such that $K(\mathbf{x}_i, \mathbf{x}_j) = \boldsymbol{\phi}(\mathbf{x}_i) \cdot \boldsymbol{\phi}(\mathbf{x}_j)$; kernel-based methods are often just linear predictors in that space.

Deep learning: definition

- A predictor that uses a hierarchy of features of the input, typically (but not always) learned end-to-end jointly with the predictor.

$$f_y(\mathbf{x}) = F_L(F_{L-1}(F_{L-2}(\cdots F_1(\mathbf{x})\cdots)))$$

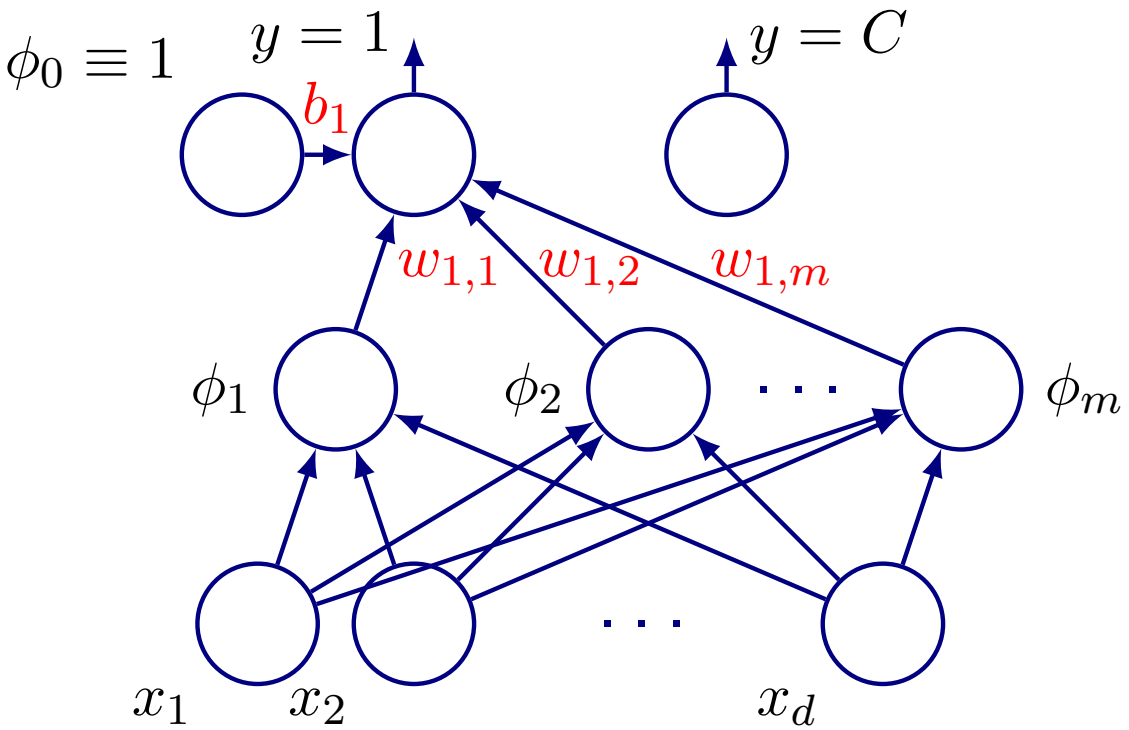
- Learning methods that are not deep:
  SVMs
  nearest neighbor classifiers
  decision trees
  perceptron
  PCA, CCA, kernel PCA, ...

Neural networks

- General form of shallow linear classifiers: score is computed as

$$f_y(\mathbf{x}; \mathbf{w}, \mathbf{b}) = \mathbf{w}_y \cdot \boldsymbol{\phi}(\mathbf{x}) + b_y$$

- Representation as a *neural network*:

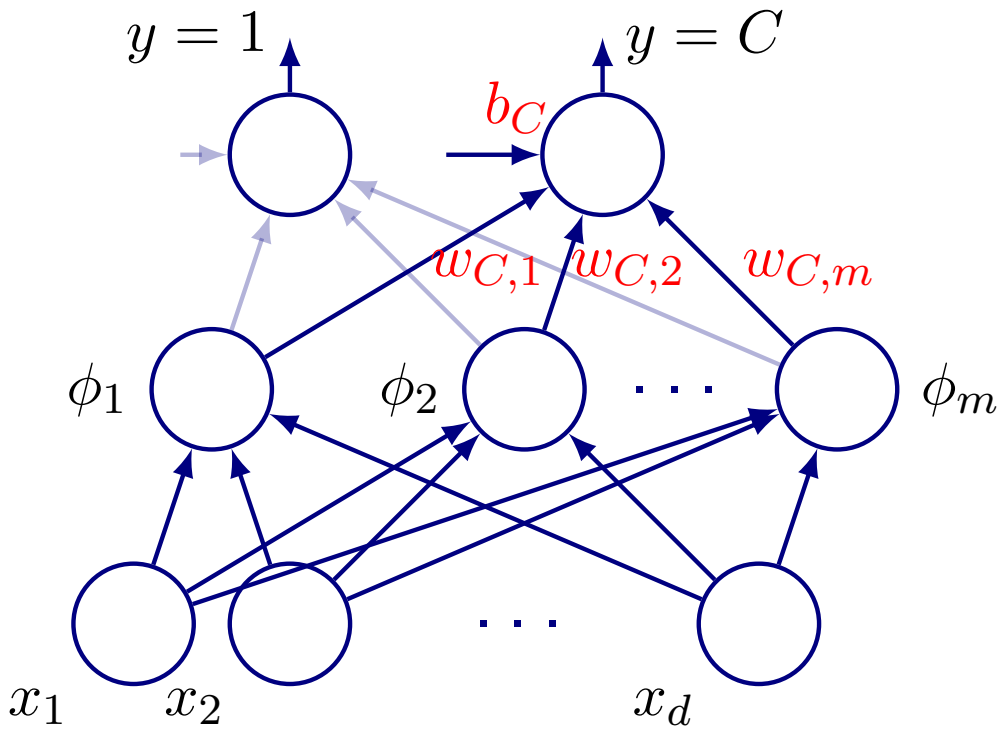


Neural networks

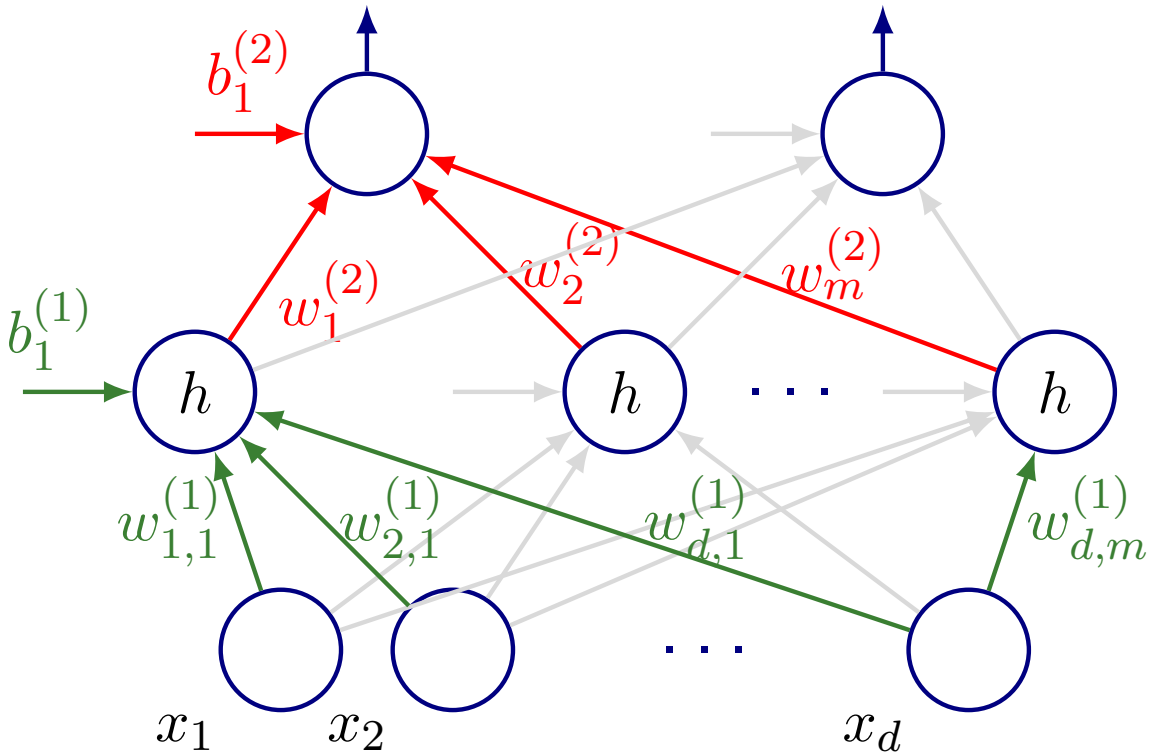
- General form of shallow linear classifiers: score is computed as

$$f_y(\mathbf{x}; \mathbf{w}, \mathbf{b}) = \mathbf{w}_y \cdot \boldsymbol{\phi}(\mathbf{x}) + b_y$$

- Representation as a *neural network*:
- Weights $\mathbf{w} = [\mathbf{w}_1, \ldots, \mathbf{w}_C]$, $\mathbf{w}_c \in \mathbb{R}^m$
- Biases $\mathbf{b} = [b_1, \ldots, b_C]$



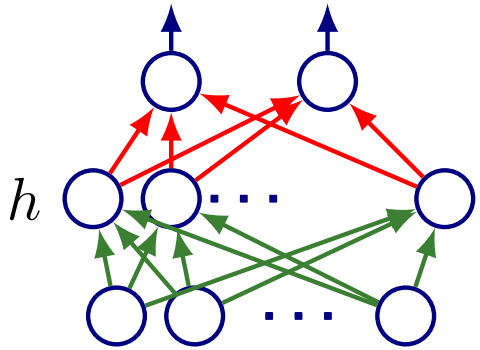
Two-layer network



- Idea: learn parametric features $\phi_j(\mathbf{x}) = h(\mathbf{w}_j^{(1)} \cdot \mathbf{x} + b_j^{(1)})$ for some nonlinear *activation function* h

Feedforward networks

- Feedforward operation, from input $\mathbf{x}$ to output $\hat{y}$:



$$f_y(\mathbf{x}) = \sum_{j=1}^{m} w_{j,y}^{(2)} h\left(\sum_{i=1}^{d} w_{i,j}^{(1)} x_i + b_j^{(1)}\right) + b_y^{(2)}$$

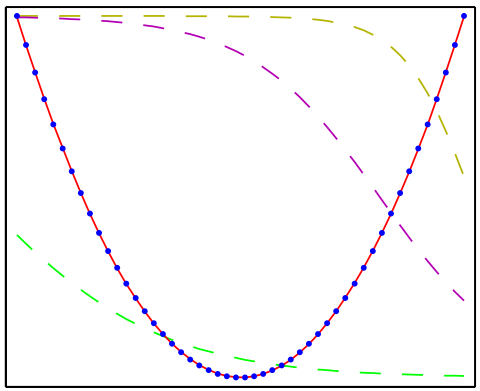
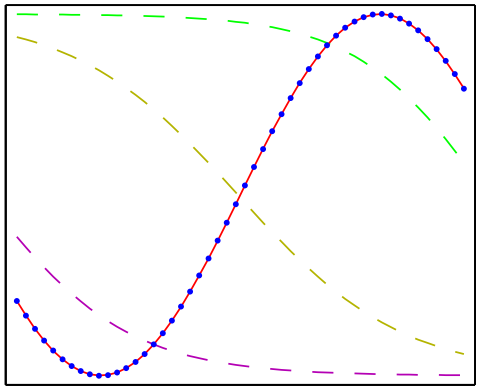
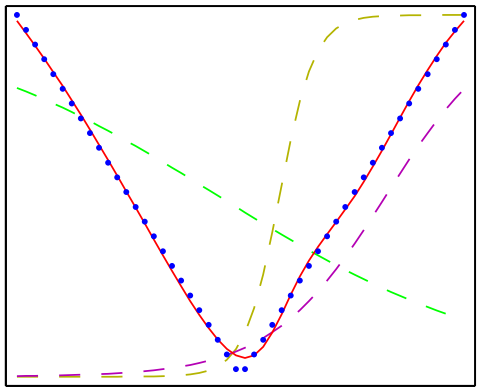
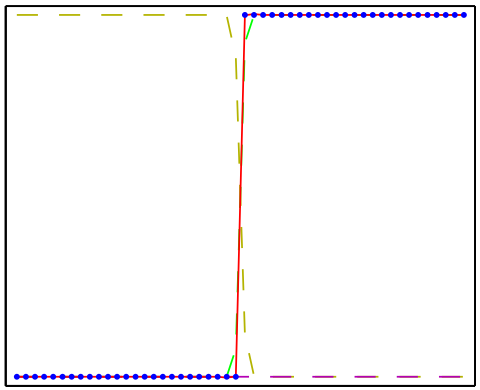
- In matrix form:

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}_2 \cdot h\left(\mathbf{W}_1 \cdot \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2$$

where h is applied elementwise; $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{W}_1 \in \mathbb{R}^{m \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{C \times m}$, $\mathbf{b}_2 \in \mathbb{R}^C$, $\mathbf{b}_1 \in \mathbb{R}^m$

Power of two layers

- Theoretical result [Cybenko 1989]: 2-layer net with linear output (sigmoid hidden units) can approximate any continuous function over compact domain to arbitrary accuracy **(given enough hidden units!)**
- Examples: 3 hidden units with $\tanh(z) = \frac{e^{2z}-1}{e^{2z}+1}$ activation

[from Bishop]

Power of more layers

- Theoretical result [Cybenko 1989]: 2-layer net with linear output (sigmoid hidden units) can approximate any continuous function over compact domain to arbitrary accuracy **(given enough hidden units!)**
- But, > 2 layers can represent a given function **more compactly**
- Example: parity of n-bit numbers, with AND, OR, NOT, XOR gates
- Trivial shallow architecture: express parity as DNF or CNF
  but need exponential number of gates!
- Deep architecture: a tree of XOR gates

Learning a neural network

- Example: Softmax classifier

$$\mathbf{f}(\mathbf{x}) = \mathbf{W}_2 \cdot h\left(\mathbf{W}_1 \cdot \mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2$$

$$\text{recall: } \hat{p}(y = c|\mathbf{x}) = \exp(f_c(\mathbf{x})) / \sum_j \exp(f_j(\mathbf{x}))$$

- Log loss computed on $\mathbf{f}(\mathbf{x})$ vs. true label y:

$$L(\mathbf{x}, y) = -\log \hat{p}(y|\mathbf{x}) = -f_y(\mathbf{x}) + \log \sum_c \exp\left(f_c(\mathbf{x})\right)$$

- Learning the network: initialize parameters, then run [stochastic] gradient descent, updating according to

$$\frac{\partial L}{\partial \mathbf{w}_2}, \quad \frac{\partial L}{\partial \mathbf{b}_2}, \quad \frac{\partial L}{\partial \mathbf{W}_1}, \quad \frac{\partial L}{\partial \mathbf{b}_1}$$

Review: Gradient descent

- Iteration counter $t = 0$
- Initialize $\Theta^{(t)}$ (to zero or a small random vector)
- for $t = 1, \ldots$:
  compute gradient on data (X, Y)

$$\mathbf{g}^{(t)}(X, Y) = \nabla_\Theta f\left(X, Y; \Theta^{(t-1)}\right)$$

  update model

$$\Theta^{(t)} = \Theta^{(t-1)} - \eta \mathbf{g}^{(t)}$$

  check for convergence (what does this mean?)
- The *learning rate* η controls the step size

Running gradient descent

- An epoch: a single pass through the training set
- Single "iteration" t is an epoch:
  loop over examples (or in parallel) computing $\mathbf{g}^{(t)}(\mathbf{x}_i, y_i)$
  accumulate the gradient

$$\mathbf{g}^{(t)}(X, Y) = \frac{1}{N} \sum_i \mathbf{g}^{(t)}(\mathbf{x}_i, y_i)$$

  make a single update at the end of the epoch.
- Assuming N is large, $\mathbf{g}^{(t)}(X, Y)$ is a good estimate for gradient, but it costs a lot to compute.

Stochastic gradient descent

- Computing gradient on all N examples is expensive and may be wasteful: many data points provide similar information
- Idea: present examples one at a time, and pretend that the gradient on the entire set is the same as gradient on one example
- Formally: *estimate* gradient of the loss on a single example

$$\frac{1}{N}\sum_{i=1}^{N}\nabla_\Theta L(y_i,\mathbf{x}_i;\Theta) \approx \nabla_\Theta L(y_t,\mathbf{x}_t;\Theta)$$

- Mini-batch version: for some $B \subset [N]$, $|B| \ll N$,

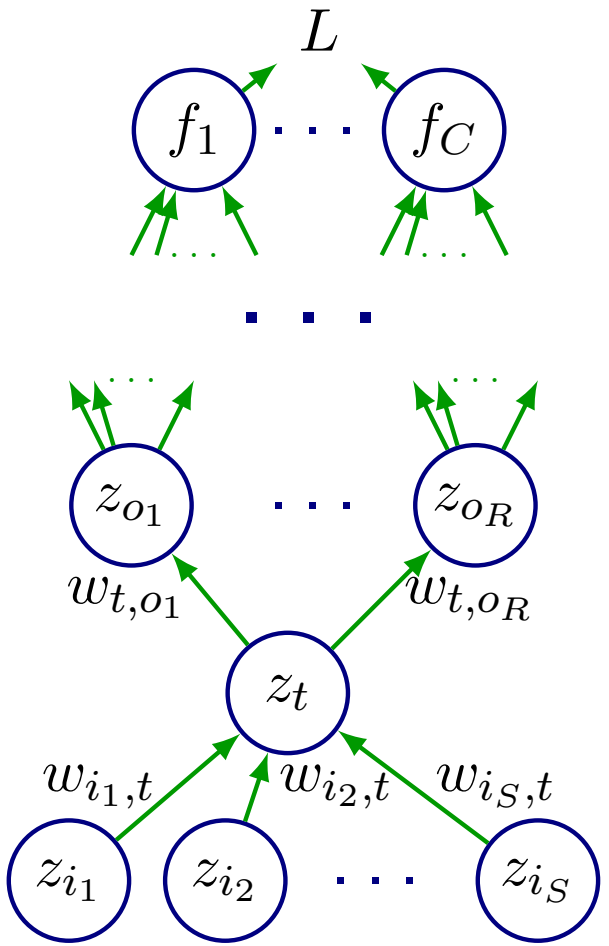
$$\frac{1}{N}\sum_{i=1}^{N}\nabla_\Theta L(y_i,\mathbf{x}_i;\Theta) \approx \frac{1}{|B|}\sum_{t\in B}\nabla_\Theta L(y_t,\mathbf{x}_t;\Theta)$$

Backpropagation

- General unit activation in a network (ignoring bias)
- Unit t receives input from $I(t) = \{i_1, \ldots, i_S\}$, sends to $O(t) = \{o_1, \ldots, o_R\}$

$$a_t = \sum_{j \in I(t)} w_{jt} z_j$$

$$z_t = h(a_t)$$



- The loss L depends on w_{jt} only through a_t

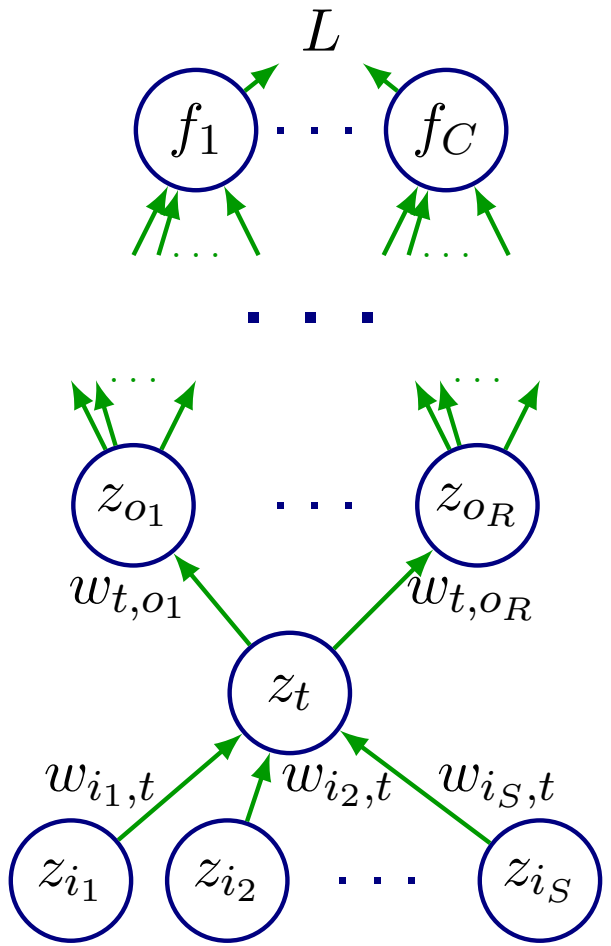
$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} \frac{\partial a_t}{\partial w_{jt}}$$

Backpropagation

- General unit activation in a network (ignoring bias)
- Unit t receives input from $I(t) = \{i_1, \ldots, i_S\}$, sends to $O(t) = \{o_1, \ldots, o_R\}$

$$a_t = \sum_{j \in I(t)} w_{jt} z_j$$

$$z_t = h(a_t)$$



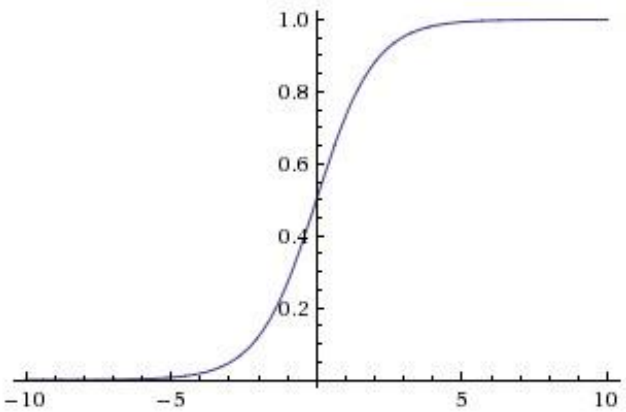
- The loss L depends on w_{jt} only through a_t

$$\frac{\partial L}{\partial w_{jt}} = \frac{\partial L}{\partial a_t} \frac{\partial a_t}{\partial w_{jt}}$$

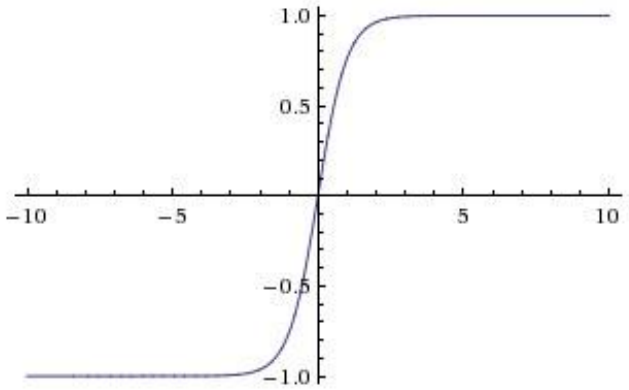
$$= \frac{\partial L}{\partial a_t} z_j$$

Choice of non-linearity: 1070s-2010

sigmoid : $h(a) = \dfrac{1}{1 + \exp(a)}$



$h(a) = \tanh(a)$



- Good: squash activations to a fixed range
- Bad: gradient is nearly zero far away from midpoint

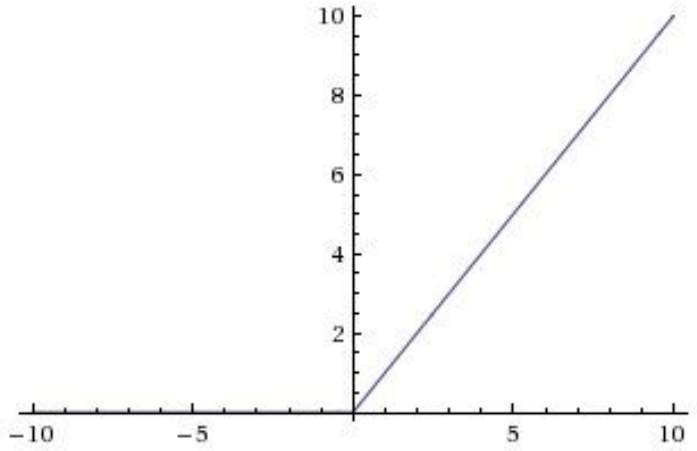
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial h(a)} \frac{dh}{da} \approx 0$$

can make learning very, very slow

2010: RELU

- Intuition: make the non-linearity non-saturating, at least in part of the range
- Rectified linear units:

$$h(a) = \max(0, a)$$



- Greatly speeds up convergence compared to sigmoid (order of magnitude)
- Often used as the "default" nonlinearity in recent work

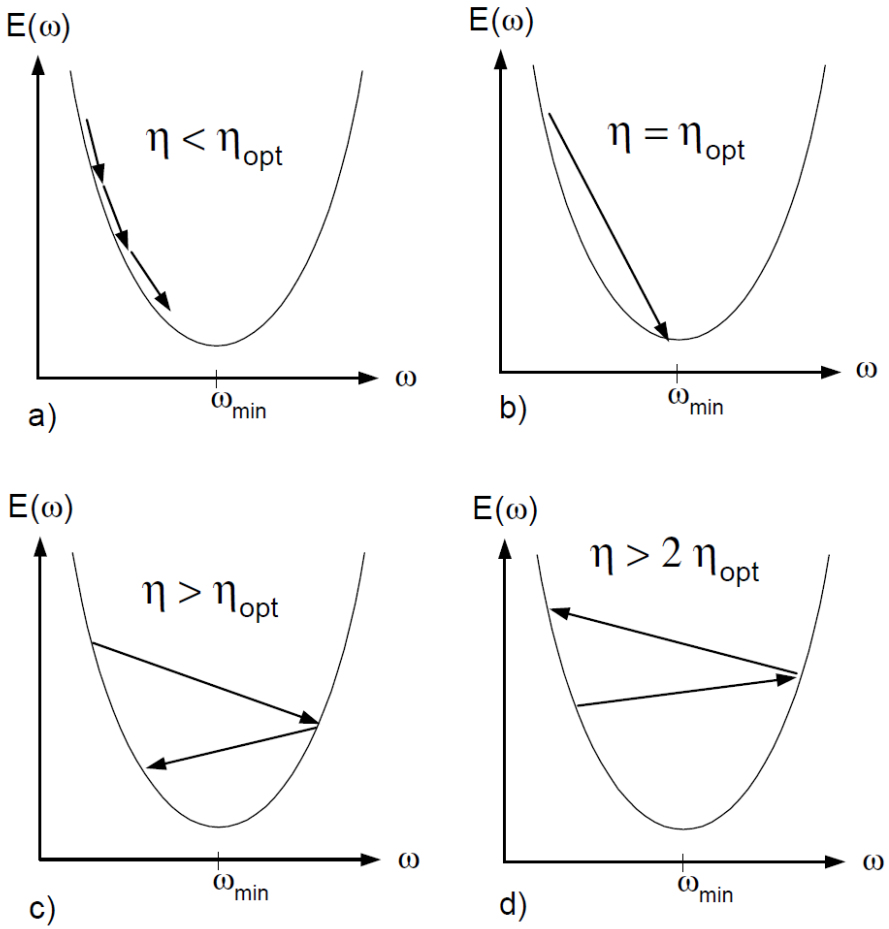
Basic stochastic gradient descent

- Learning hyperparameters: architecture, regularizer R
- Hyperparameters: learning rate η, batch size B
- Initialize weights and biases
- Each epoch: shuffle data, partition into batches, iterate over batches b

$$\mathbf{w} = \mathbf{w} - \eta \left[\nabla_{\mathbf{w}} L(X_b, Y_b) + \nabla_{\mathbf{w}} R(\mathbf{w}) \right]$$

Learning rate

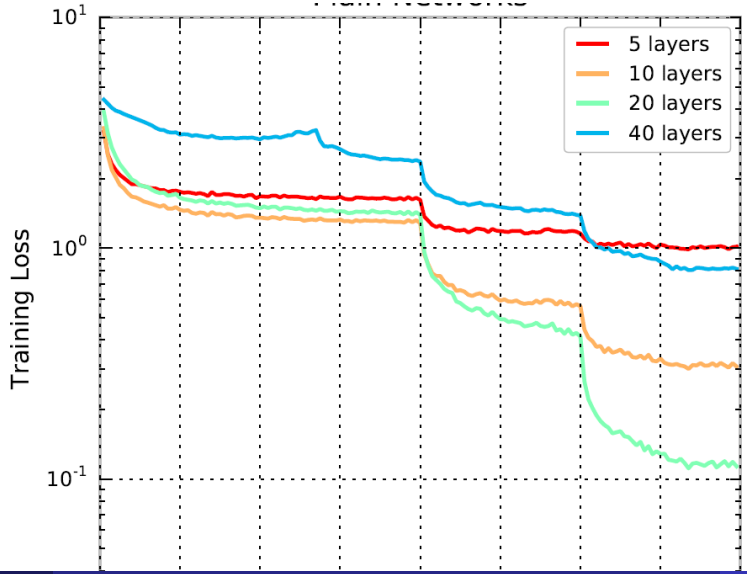
- Generally, for convex functions, gradient descent will converge
- Setting the learning rate η may be very important to ensure rapid convergence



from Lecun et al, 1996

Learning rate schedules

- Generally, we want the learning rate to decay with training time
- Could set up automatic schedule, e.g., drop by a factor of α every β epochs.
- Most common in practice: some babysitting...
  - start with a resonable learning rate
  - monitor training loss
  - drop LR (typically 1/10) when learning appears stuck
- Monitoring training loss: Too expensive to evaluate on entire training set frequently; instead, use rolling average of mini-batch loss value



AdaGrad

There is a variety of optimizers that have been proposed to automatically adjust learning rates. E.g. AdaGrad:

- Intuition [Duchi et al.]: parameters (directions of the parameter space) are updated with varying frequency
- Idea: reduce learning rate in proportion to the updates
- Maintain cache s_i for each parameter θ_i; when updating,

$$s_i = s_i + \left(\frac{\partial L}{\partial \theta_i}\right)^2$$

$$w_i = w_i - \eta \frac{\partial L}{\partial \theta_i} / \left(\sqrt{s_i} + \epsilon\right)$$

RMSprop

- Modified idea from AdaGrad; ["published" in Geoff Hinton's Coursera slides]
- Cached rate allows for "forgetting"

$$s_i = \delta s_i + (1-\delta)\left(\frac{\partial L}{\partial \theta_i}\right)^2,$$
$$w_i = w_i - \eta \frac{\partial L}{\partial \theta_i} / \left(\sqrt{s_i} + \epsilon\right)$$

- The decay rate δ is typically 0.9–0.99

Adam optimizer

- Kind of like RMSprop with momentum [Kingma et al.]
- First order momentum update for w_i:

$$m_i = \beta_1 m_i + (1 - \beta_1) \frac{\partial L}{\partial \theta_i}$$

- Second order:

$$v_i = \beta_2 v_i + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta_i} \right)^2$$

- Parameter update:

$$w_i = w_i - \eta \frac{m}{\sqrt{v} + \epsilon}$$

Regularization: Weight decay

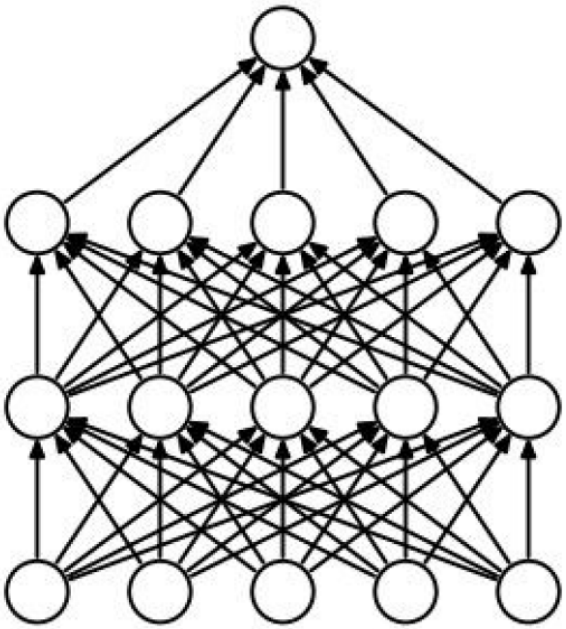
- In neural networks, L_2 regularization is often called weight decay
- Easy to incorporate weight decay into the gradient calculation

$$\nabla_{\mathbf{w}} \lambda \|\mathbf{w}\|^2 = 2\lambda \mathbf{w}$$

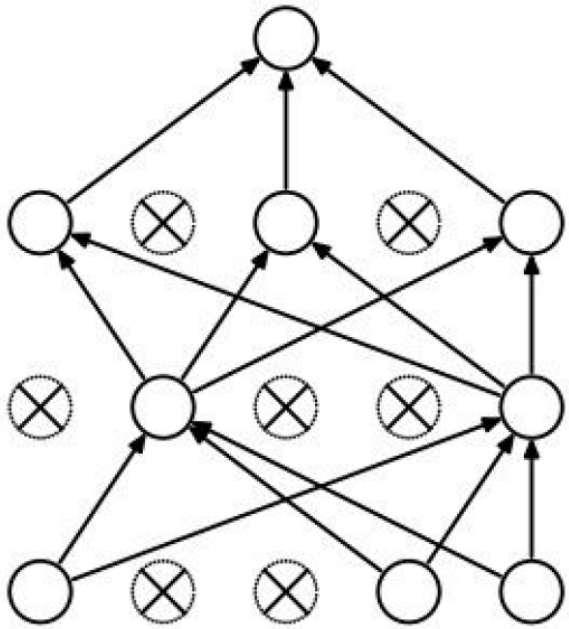
- One more hyperparameter λ to tune
  with large data sets, typically seems inconsequential

Dropout

- Part of overfitting in a neural net: unit co-adaptation
  idea: prevent it by disrupting co-firing patterns
- Dropout [Srivastava et al]: during each training iteration, randomly "remove" units, just for that update



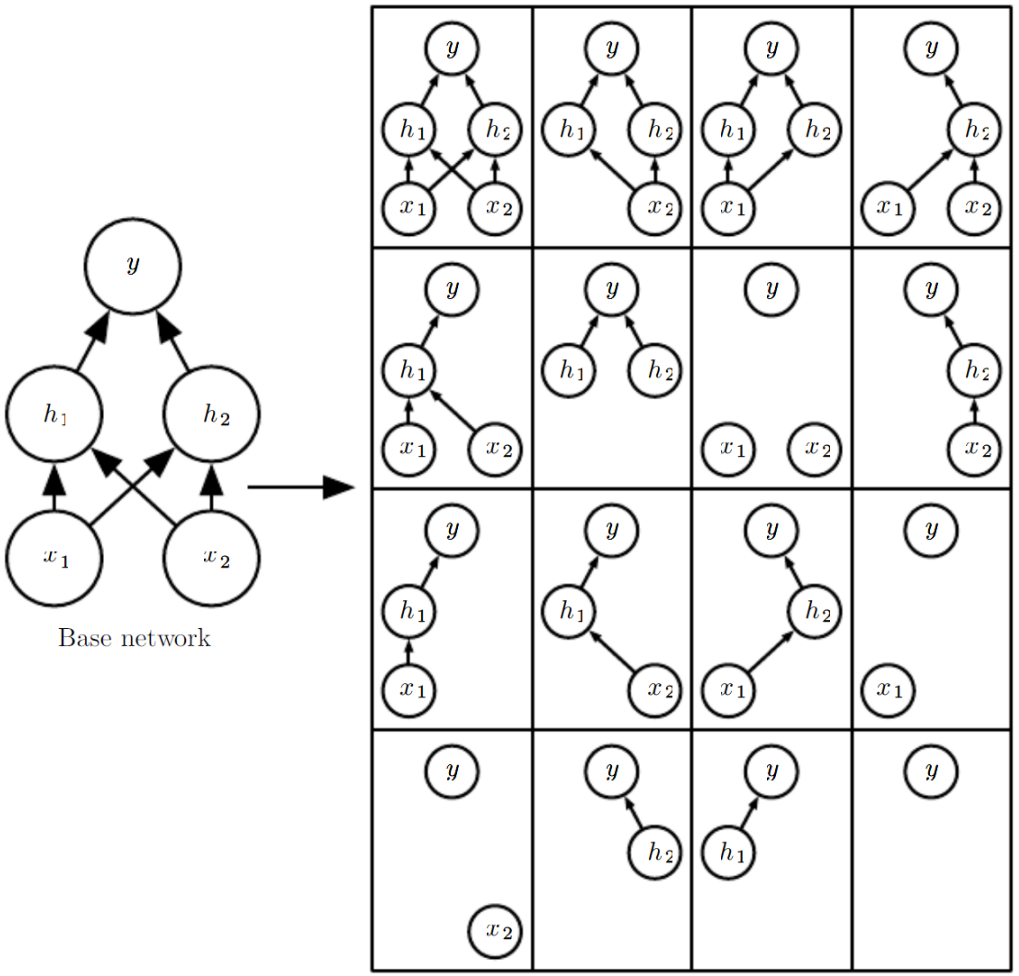
(a) Standard Neural Net



(b) After applying dropout.

Dropout as regularizer

- With each particular dropout set, we have a different network
- Interpretation: training an ensemble of networks with shared parameters



[Goodfellow et al.]

Unsupervised deep representation learning

Examples

- Use neural network as a form of out-of-sample extension with one of the projection methods we have discussed
- Autoencoders
- Probabilistic generative models that double as learned representations (have we seen this before?)
  - Restricted Boltzmann machines, deep belief networks (we will probably not cover these)
  - Variational autoencoders
  - Future prediction models (including language models)
- Self-supervised representation learning

Parametric t-SNE

- An example of the first type of deep representation learning: Use a neural network as a form of out-of-sample extension for t-SNE
- Just like t-SNE, but with a built-in out-of-sample extension

$$
\begin{aligned}
q_{ij} &= \frac{(1+||\mathbf{y}_i - \mathbf{y}_j||^2)^{-1}}{\sum_{k \neq l}(1+||\mathbf{y}_k - \mathbf{y}_l||^2)^{-1}} \\
\longrightarrow q_{ij} &= \frac{(1+||f(\mathbf{x}_i) - f(\mathbf{x}_j)||^2)^{-(\alpha+1)/2}}{\sum_{k \neq l}(1+||f(\mathbf{x}_k) - f(\mathbf{x}_l)||^2)^{-(\alpha+1)/2}}
\end{aligned}
$$

- $f(\cdot)$ is a deep neural network
- α is the number of degrees of freedom of the Student t-distribution
- (The higher the dimensionality of $f(\cdot)$, the higher the number of degrees of freedom)

Autoencoders

- One neural network $f(\mathbf{x})$ (**encoder**) produces a learned representation of the input
- A second neural network $g(\mathbf{x})$ (**decoder**) tries to reproduce the input
- Typical loss for continuous input: $||g(f(\mathbf{x})) - \mathbf{x}||^2$

Autoencoders

- As described, we could just learn an identity mapping...
- Only makes sense if we constrain $f(\cdot)$ somehow

Autoencoders

- As described, we could just learn an identity mapping...
- Only makes sense if we constrain $f(\cdot)$ somehow
  - Limit dimensionality (number of units at the output) of $f(\mathbf{x})$

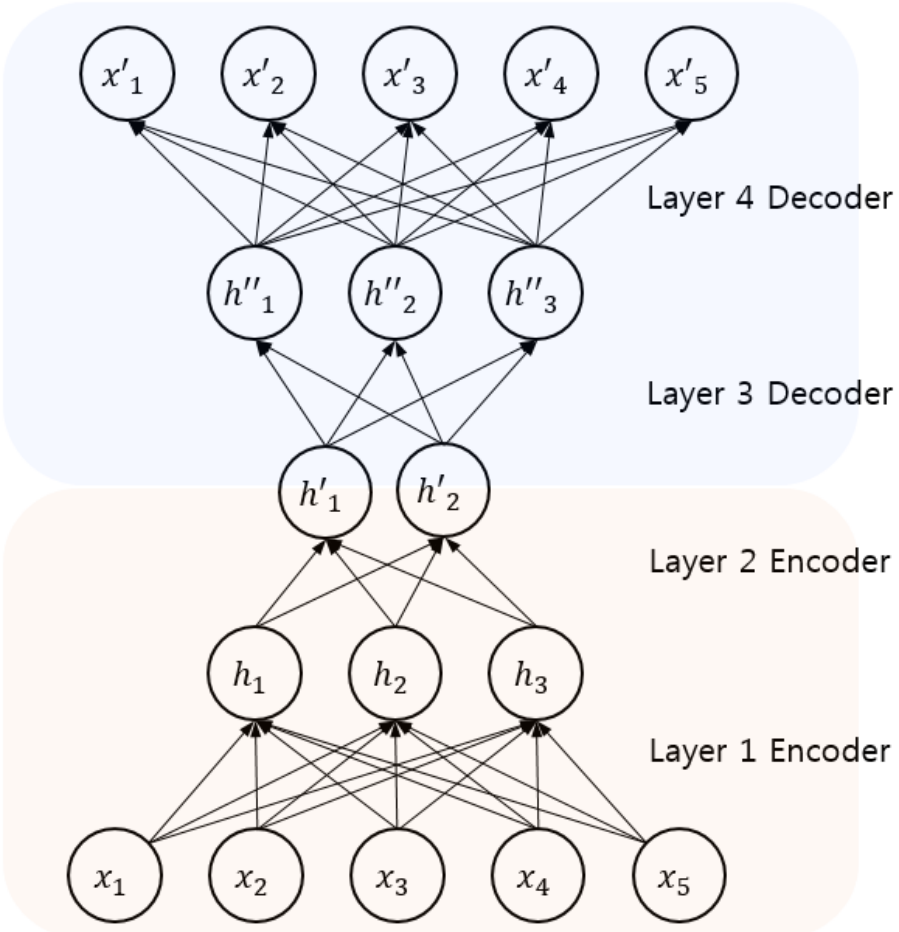
Autoencoders

- As described, we could just learn an identity mapping...
- Only makes sense if we constrain $f(\cdot)$ somehow
  - Limit dimensionality (number of units at the output) of $f(\mathbf{x})$
  - Add a penalty to the loss, e.g. sparsity (ℓ_0, ℓ_1)

Autoencoders

- As described, we could just learn an identity mapping...
- Only makes sense if we constrain $f(\cdot)$ somehow
  - Limit dimensionality (number of units at the output) of $f(\mathbf{x})$
  - Add a penalty to the loss, e.g. sparsity (ℓ_0, ℓ_1)
- Denoising autoencoders: Add noise to the input, but try to reproduce the *clean* input

Autoencoders



A note about autoencoder losses

- Autoencoders can be viewed as representing a distribution $p(\mathbf{x}|\mathbf{z})$ over the encoder input $\mathbf{x}$ given the output $\mathbf{z} = f(\mathbf{x})$
- If $p(\mathbf{x}|\mathbf{z})$ is, say, a spherical Gaussian with mean $g(\mathbf{z})$, then optimizing the likelihood over a training set is equivalent to the usual squared loss
- Other distributions produce different losses...

Variational autoencoders

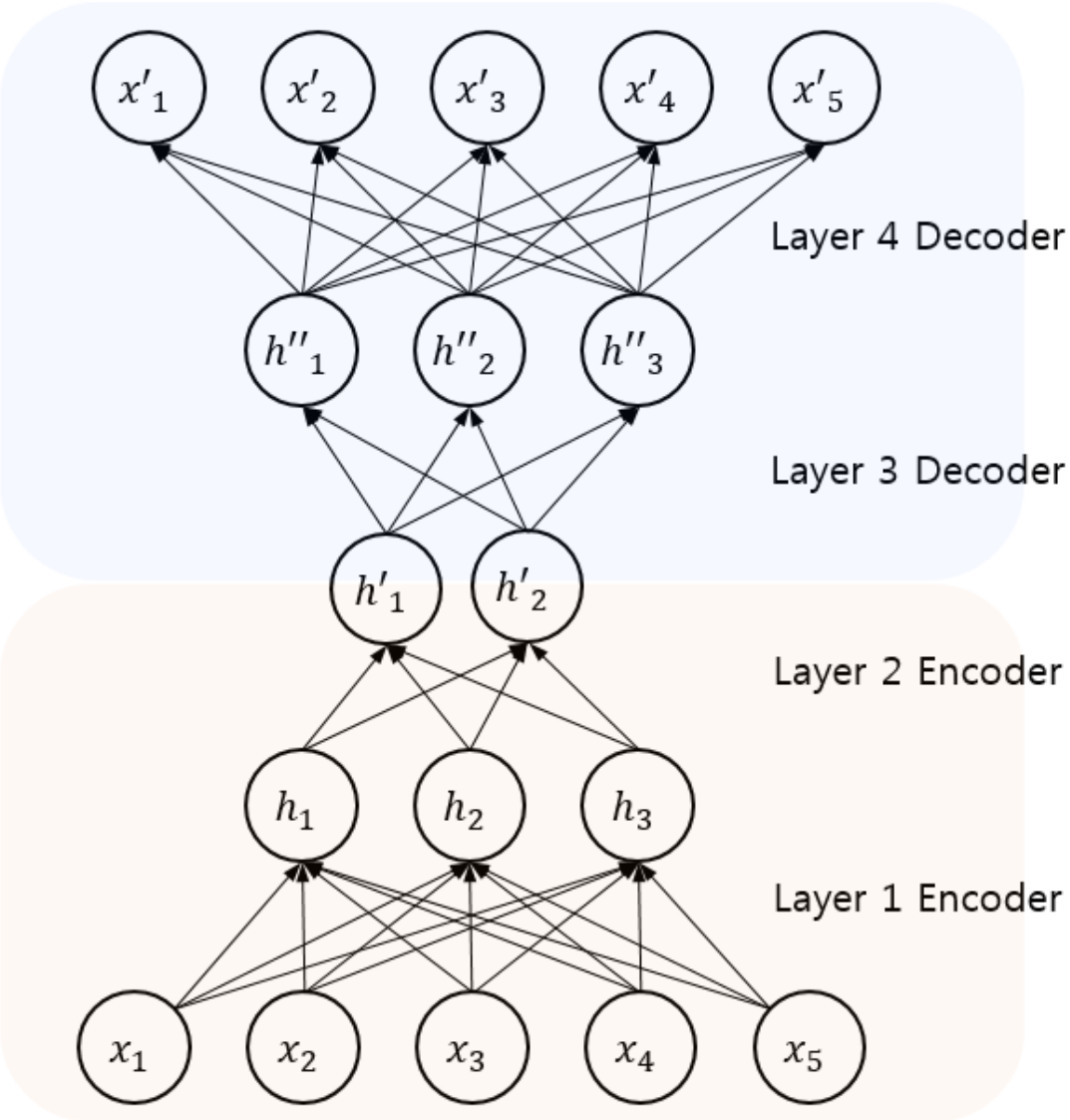
Variational autoencoders

- More or less simultaneously proposed by Kingma & Welling (2013) and Rezende *et al.* (2014)
- Popular, fast, and relatively easy to train
- Generative model for a vector of random variables $\mathbf{x}$ assumed to be generated from a set of latent variables $\mathbf{z}$: $p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x}|\mathbf{z})p(\mathbf{z})d\mathbf{z}$
- For today, assume $\mathbf{x}, \mathbf{z}$ are both continuous
- No "meaning" to $\mathbf{z}$ (unlike in some other latent variable models that we'll get to later)
- Initially used for generation, but has also been successful for representation learning

Variational autoencoders for density estimation

- "Autoencoders" because they compute a density of $\mathbf{z}$ given $\mathbf{x}$ via an *encoder* and a density of $\mathbf{x}$ given $\mathbf{z}$ via a *decoder*
- "Variational" because they involve approximating a density via optimization

Reminder: Autoencoders

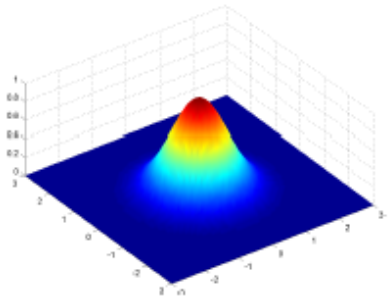


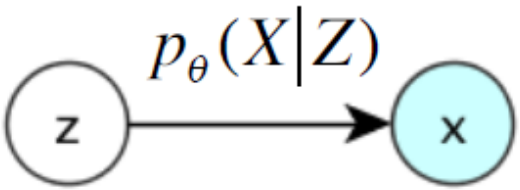
[Dürr 2016]

VAE generation model (decoder)

$z \sim p(z)$ multivariate Gaussian

$x|z \sim p_\theta(x|z)$



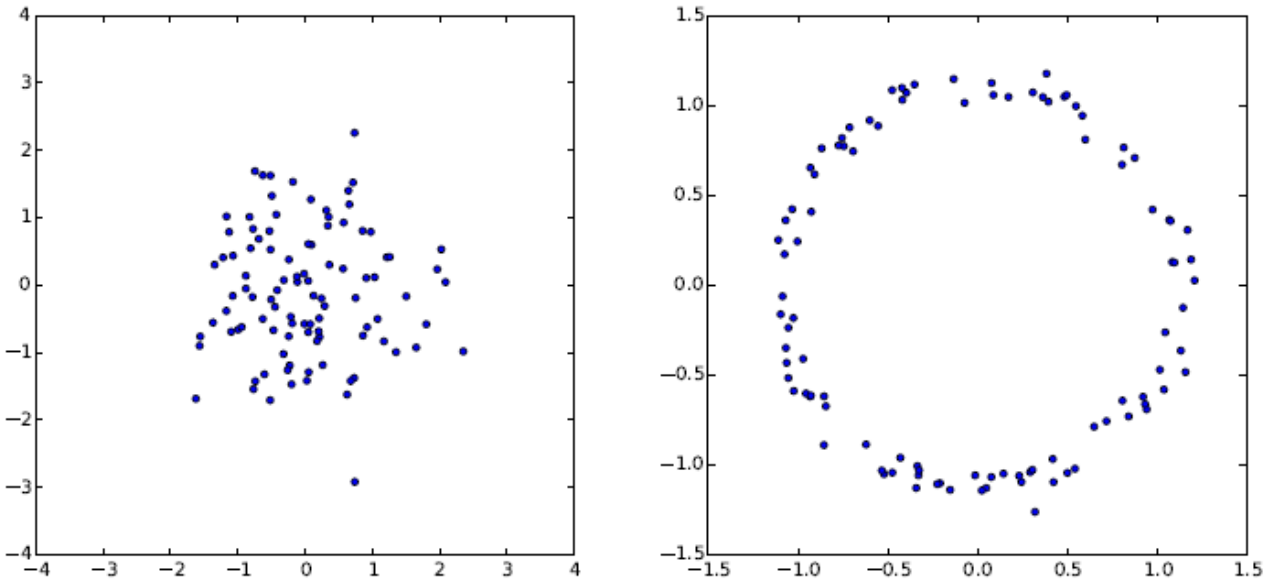




[Dürr 2016]

VAE generation model (decoder)

Key insight: Any d-dimensional distribution can be generated by taking d normally distributed variables and mapping them through some appropriate (possibly very complicated) function

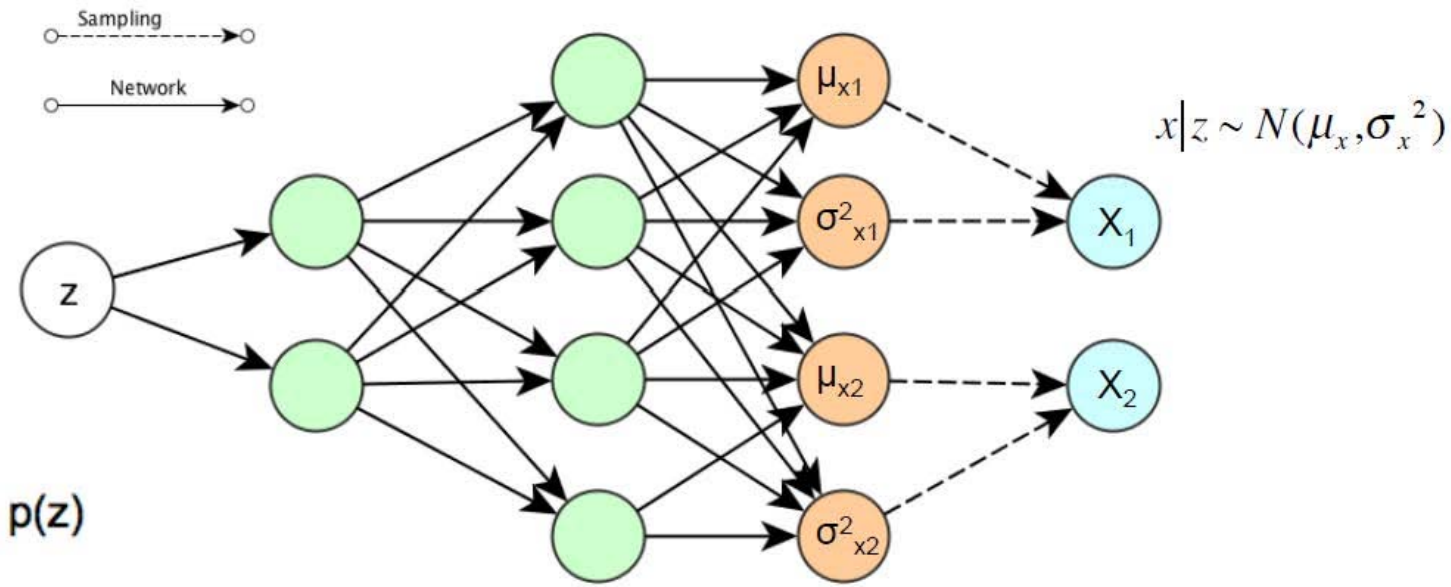


[Doersch 2016]

Here $x = g(z) = z/10 + z/||z||$

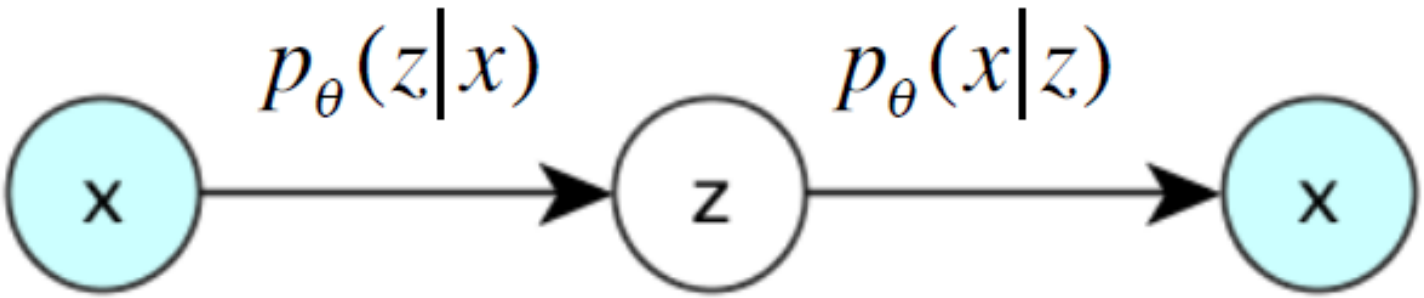
VAE neural decoder

- Assume w.l.o.g. that latent variable $\mathbf{z}$ is 1-dimensional, $\mathbf{x}$ is 2-dimensional
- For now, assume that $\mathbf{z}$ is Gaussian, $\mathbf{x}$ is diagonal Gaussian



[Dürr 2016]

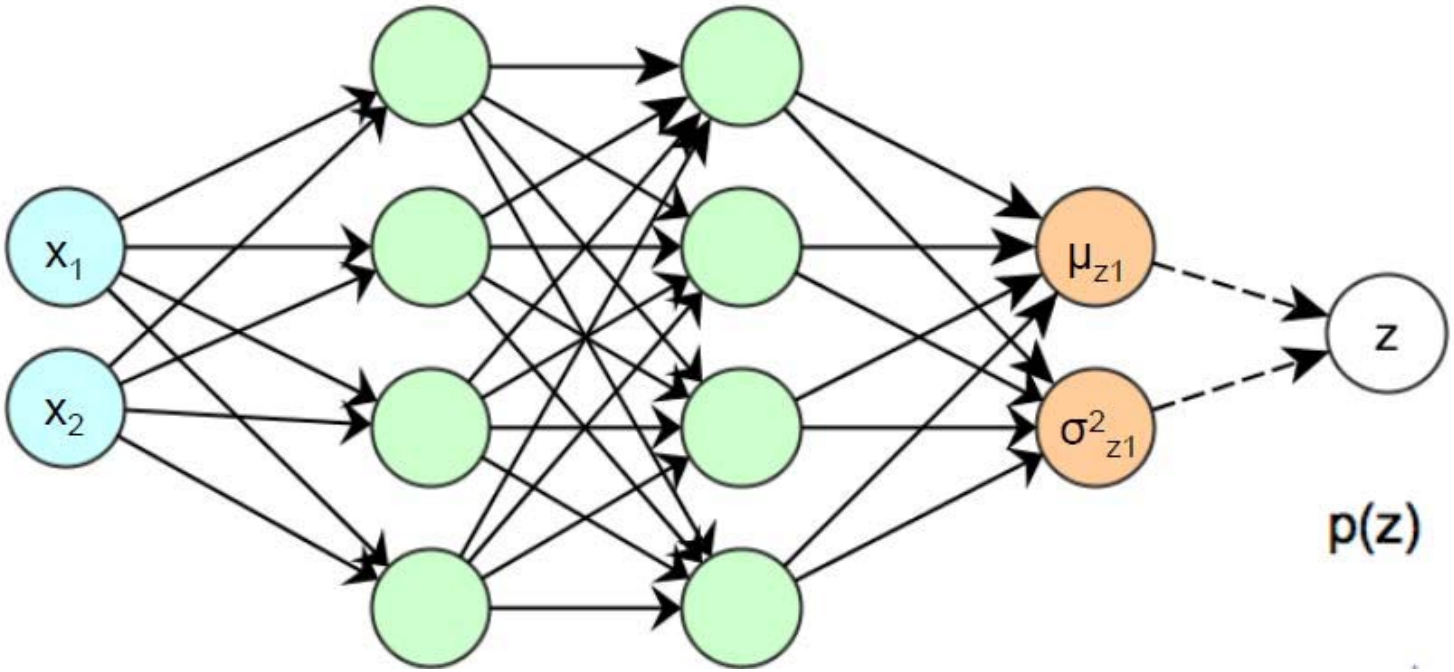
VAE encoder-decoder model



[Dürr 2016]

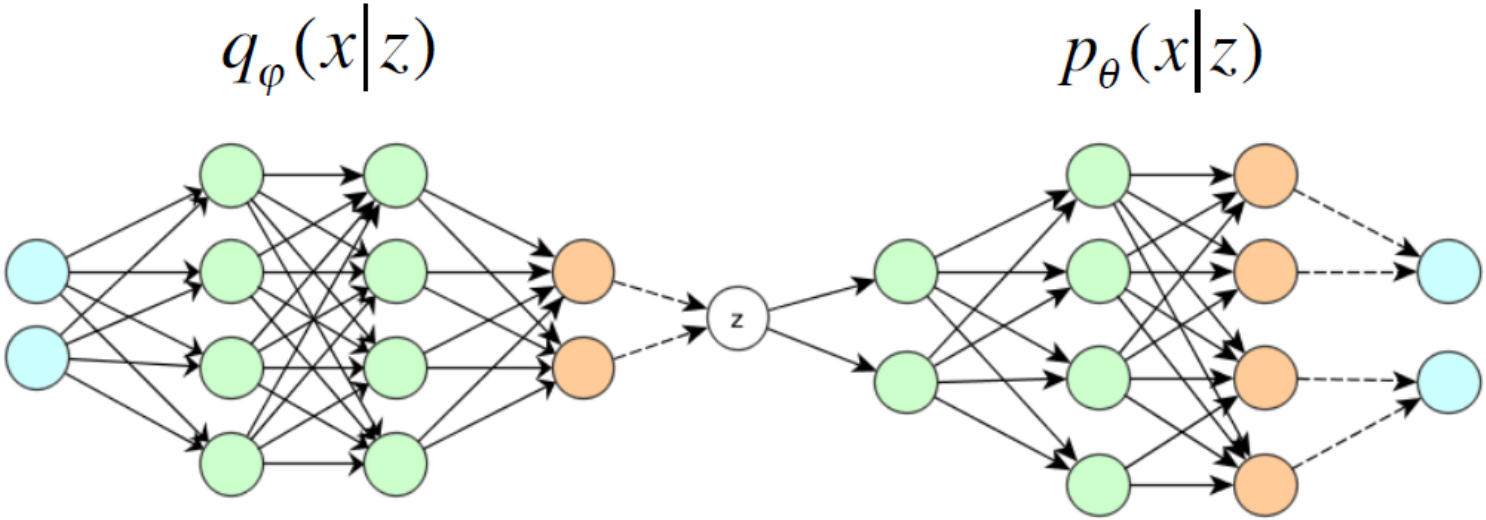
- But $p(\mathbf{z}|\mathbf{x})$ can be very costly to estimate
- We will approximate it with another neural function $q(\mathbf{z}|\mathbf{x})$
- That function $q(\mathbf{z}|\mathbf{x})$ is the encoder

VAE neural encoder



[Dürr 2016]

Complete VAE



[Dürr 2016]

Both a learned representation and a generative model!

- Where is the learned representation?
- How do we generate from this model?